Applying automatic differentiation to derivatives pricing and risk: practical experience from production

Mihail Andreev Alexander Lykov Rauil Khabibullin Valery Tyan

Sber CIB

October 2026

The talk we were looking for in 2022

We migrated a production derivatives pricing engine from one automatic differentiation (AD) engine to another: from a **proprietary third-party AAD library** to **libtorch**, the C++ API of PyTorch.

- Large C++ codebase, in continuous development, numbers used by the business every day
- The difficulty of the project was **technical**, not mathematical
- The migration was completed without rewriting the engine from scratch

Why we are giving this talk

When we started, we wanted exactly this kind of material and could not find it. We searched the literature, wrote to people, and talked to them in person. The theory of AD is very well documented. Migrating a live production library is not.

Outline

1. Motivation: why a pricing engine needs derivatives
2. AD: a longer history than some people assume
3. Complexity bounds, and why AD is still not free
4. The tool landscape, and who we asked
5. Our constraints and the decision
6. Implementation and results

Motivation

Why a pricing engine needs derivatives

The pricing engine

context

Sber CIB runs an in-house C++ engine for pricing OTC derivatives. It evolves continuously with the needs of the business.

What changed after February 2022:

- Vendors left the market; import substitution became mandatory
- A sharp increase in the amount of work on the engine
- More downstream systems consuming pricing and risk
- New scenarios and new calculation types to support

Demand for throughput and for risk granularity grows faster than the hardware budget.

Greeks: the computational problem

The price is an expectation of discounted cash flows under $\mathbb{Q}$, depending on a vector of market inputs $\theta = (\theta_1, \ldots, \theta_n)$:

$$V(\theta) = \mathbb{E}^{\mathbb{Q}}\left[\sum_i D(0, T_i)\,\mathrm{CF}_i(\theta)\right]$$

Greeks are the partial derivatives $\partial V / \partial \theta_k$, $k = 1, \ldots, n$.

Where n becomes large:

- Point-wise curve risk instead of a parallel shift
- Derivatives on a basket of underlyings
- CVA and other xVA: sensitivity to *every* curve, surface and spread in the netting set, easily thousands of inputs

Finite differences

the baseline, and our fallback

$$\frac{\partial V}{\partial \theta_k} \approx \frac{V(\theta + h\,e_k) - V(\theta - h\,e_k)}{2h}$$

	Finite differences	Adjoint AD
Cost of n Greeks	$\sim n$ valuations	$\sim$ one valuation
Accuracy	truncation error, choice of h	analytic
Memory	negligible	records a tape
Adoption	no code changes	code must be restructured

For us finite differences remain the **fallback**. Falling back costs time and risk granularity: a parallel curve shift instead of point-wise risk. It is also the independent oracle we reconcile AD numbers against.

Automatic differentiation

History, complexity, and what it does not solve

A longer history than some people assume

1953	Nolan, MSc thesis, MIT: analytical differentiation on a digital computer
1959	**Beda, Korolev, Sukhikh, Frolova: AD programs for the BESM computer (ITM & VT, Academy of Sciences, Moscow) — earliest known implementation**
1959–60	Rozonoer; Boltyanskii, Gamkrelidze, Pontryagin: the maximum principle is the reverse mode in continuous time
1964	Wengert, CACM: forward mode as a general method
1970, 1976	Linnainmaa: first published description of the reverse mode
1974	Werbos: discrete-time formulation, later known as backpropagation
1980	Speelpenning, PhD: first genuinely automatic reverse mode
1983	Baur and Strassen: the "cheap gradient" complexity result
1986	Rumelhart, Hinton, Williams: backpropagation enters mainstream ML
1989+	Griewank and others revive AD in computer science

The idea was discovered and forgotten several times, and by 1990 it was established in computer science but used by almost nobody. Two fields then picked it up, independently.

Two branches, and the one that helped us was not ours

In quantitative finance

- **2006** Giles and Glasserman, "Smoking adjoints": AD arrives in finance
- **2008–12** Capriotti; Capriotti and Giles: AAD as a design paradigm rather than a tool
- **2014–15** Andreasen, "CVA on an iPad mini"; Danske Bank wins Risk In-House System of the Year
- **2018–19** Savine: textbooks aimed at practitioners
- **2024** Capriotti and Giles: 15 Years of Adjoint Algorithmic Differentiation in Finance

Purpose-built, by a small field for itself.

In machine learning

- **1986+** the same reverse mode, under the name backpropagation, becomes the way networks are trained
- **2010s** reverse-mode AD moves from hand-coded gradients into the core of every training framework
- **2015–16** Theano, TensorFlow, then PyTorch: production AD engines, written for someone else's problem
- **today** general purpose: they differentiate arbitrary user code written within their constraints, and second-order gradients are a routine requirement

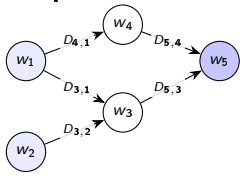
Where the two branches met, for us

libtorch exists because an entire industry needed reverse-mode AD on tensors. As a side effect comminuty now has a production-grade AD engine, with its documentation, testing and capabilities, without funding any of it. We also inherited its design centre, which is not scalar pricing arithmetic.

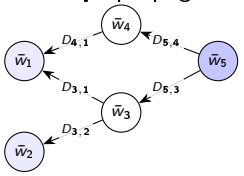
The adjoint mode in one picture

A program is a sequence of elementary assignments $w_i = \varphi_i(\{w_j\}_{j \prec i})$. Each arc of its computational graph carries a *local* derivative $D_{i,j} = \partial \varphi_i / \partial w_j$.

Forward sweep: evaluate, record the arcs



Reverse sweep: propagate adjoints



$$\bar{w}_i = \sum_{j\,:\,i \prec j} \bar{w}_j \, D_{j,i}, \qquad \bar{w}_5 = 1 \quad \Longrightarrow \quad \bar{w}_1 = D_{3,1}\bar{w}_3 + D_{4,1}\bar{w}_4 \; = \; \frac{\partial w_5}{\partial w_1}$$

The adjoint of a node is the sum of the adjoints of everything that depends on it, weighted by the local derivatives. One backward pass fills in every input.

Following Capriotti and Giles (2024), Figs. 1 and 3.

What that buys, and what we use

For $F : \mathbb{R}^n \to \mathbb{R}^m$, under the standard complexity model:

$$\frac{\mathrm{Cost}[F \,\&\, J]}{\mathrm{Cost}[F]} \;\leq\; \omega_A \, m$$

The cheap gradient principle

One scalar output, $m = 1$: the *entire* gradient costs a small constant times one valuation, **independently of** n. Bumping and the forward mode both scale like n.

What we use. The adjoint mode, with the portfolio value or an aggregate as the single scalar output, and the market inputs as the n.

$\omega_A \in [3, 4]$ in Capriotti and Giles (2024)

AD is not free: the forward sweep has to be remembered

The adjoint mode needs information from the forward sweep in order to run backwards. That requirement, not the CPU, is what actually limits AD.

Two remedies are as old as the field:

- **Path-wise differentiation** in Monte Carlo. Differentiate one path, accumulate, discard the recording, repeat. Recordings stay small and the work parallelises across paths
- **Checkpointing.** Recompute parts of the forward sweep instead of storing them, trading CPU for memory. Optimal schedules are a classical result

What is *not* a property of AD itself

Textbook estimates of the form "a tape grows by several GB per second of computation per core" describe **one particular recording strategy**: a hand-written tape holding one entry per scalar operation. How much the forward sweep costs to remember is an implementation choice, and implementations differ by orders of magnitude.

What AD does not solve

the practitioner's list, as of 2019

- **AD buys speed, not accuracy.** AD and bump sensitivities are almost indistinguishable → revisited
- **It differentiates the code, not the model.** AD is exact for the function as implemented, discretisation included. Where the discretisation is not smooth in the parameter, AD is exactly right about the wrong thing
- **Control flow is not differentiated.** Digitals and barriers are not differentiable at all. The fix belongs to the valuation library (smoothing), not to the AD library
- **Constant time only in the number of inputs.** An itemised risk report over m trades is linear in m
- **One order of acceleration only.** "AD over AD" is not viable in practice, so second-order Greeks come from bumping over AD → revisited
- **Small n is not worth it.** Below a handful of inputs, bumping wins

Source: Savine (2019), Ch. 11.

The search and constraints

Tool landscape, and who we asked. What we actually had to optimise for

What technology was available to us

Our starting point in 2022. The engine already used AD, through a proprietary third-party AAD library. We had to replace it for licensing reasons, not technical ones.

Category	Examples	Character
Proprietary AAD libraries	vendor products for quant finance	purpose-built; vendor risk
Open-source C++ AD by overloading	ADOL-C, CppAD, Adept, CoDiPack, dco/c++, XAD	mature, narrow user base
Compiler-level AD	Enzyme (LLVM IR)	language-agnostic, young
Other languages	Zygote.jl, JAX	not applicable to a C++ engine
ML/AI frameworks	libtorch, TensorFlow	huge ecosystem, AD built for a different purpose

What changed over 15–20 years. Savine's generation built their own AAD engine. Then specialised commercial products appeared. Today general-purpose open-source engines are powerful. In 2022 it was not obvious that they were powerful *enough* for our codebase.

Who we contacted

~20 academic groups

- Lomonosov MSU, Mechanics and Mathematics
- Lomonosov MSU, Faculty of Chemistry
- MSU CMC, Computational Methods
- MSU CMC, System Programming
- MIPT
- HSE University
- Skoltech
- Innopolis University, Kazan
- Dorodnicyn Computing Centre, RAS
- Keldysh Institute of Applied Mathematics, RAS
- Ivannikov Institute for System Programming, RAS
- Institute for Systems Analysis, RAS
- Institute of Informatics Problems
- Institute of Control Sciences, RAS
- Russian Quantum Center
- Institute of Computational Mathematics and Mathematical Geophysics, SB RAS, Novosibirsk
- St. Petersburg Institute for Informatics and Automation, RAS

We also searched the literature and spoke to practitioners directly.

Selection criteria

Performance is not the only axis, and for us it was not the deciding one.

1. **Embeddability.** How much of an existing, template-heavy C++ codebase has to change
2. **Numerical accuracy.** Results must reconcile against the numbers the business already uses
3. **Performance in absolute terms.** Complexity bounds say the ratio is bounded. They say nothing about the constant, and AD carries real CPU and memory overhead
4. **Scalability.** Threads, cores, and memory per calculation
5. **Licensing.** The reason we were doing this at all
6. **Ecosystem.** Documentation, community, longevity, and how much help a developer can get. A widely used library is also one that AI coding assistants actually know

Replacing calculations under a live business

The engine's numbers are used in the business. A new implementation has to reproduce what the old one produced, and every difference has to be explained.

- **Moving target.** Functional development continued throughout: new models, new products, new calculation types, regular releases
- **A rewrite from scratch was rejected.** Very expensive, and with no credible completion date

The decision that made the project feasible

Adapt the code to support **two AD engines at the same time**, and migrate calculations *incrementally*: one business scenario, one set of models and algorithms at a time. Keep the **bump fallback** available throughout, so that a technical problem costs performance rather than business continuity.

The Enzyme pilot

why we started there

What it is. Enzyme differentiates LLVM IR after the optimiser has run, so it differentiates any language that compiles through LLVM.

Why it was our first candidate:

- Our engine is already heavily templated C++, so a compiler-level tool looked like *native* technology.
- Moving work to compile time was attractive in itself. We doubted that any open-source *run-time* engine could reach the performance we needed

The pilot worked

We priced and differentiated one product type end to end.

The Enzyme pilot

why we stopped

Severity	What we ran into
Critical	**Maintainability.** Writing and modifying code in this mode was markedly harder. Template-related errors routinely failed to point at the root cause, and the day-to-day development experience differed substantially from the one our team works in
Critical	**Reconciliation.** Agreeing the numbers between the two versions proved to be hard. We deliberately reused the existing implementations of the main algorithms in the prototype, and the two still came out substantially different
Moderate	**Compile time.** A real risk of uncontrolled growth, with no obvious ceiling
Minor	**Toolchain lock-in.** Clang only, and the build becomes sensitive to the compiler version

Note that neither critical item is about derivatives. Both are about living with the resulting code: whether a team can maintain it, and whether its numbers can be signed off against the numbers already in use.

We do not claim Enzyme cannot work, only that we could not make it work within our constraints. And our second assumption was wrong as well: a run-time engine did meet the performance requirement.

Implementation

What had to change in the code

The engine's code

Structure. Models, product logic and market data handling are kept separate.

Stack. C++ with heavy use of templates; Eigen for linear algebra; a proprietary AAD library until 2022.

The differentiable scalar type runs through everything: models, products, numerical algorithms. Changing the AD engine is therefore not a local edit.

The choice: libtorch

What we chose. The C++ API of PyTorch.

What it demands. Every differentiable computation must run on `torch::Tensor`. Formally this means replacing Eigen with Tensor across every algorithm and class in the engine.

Why it was acceptable anyway. The AD engine is maintained by a very large community for reasons that have nothing to do with us; licensing is permissive and stable; documentation is abundant; and the same machinery is exercised daily by an entire industry.

How it records. One node per tensor operation, holding what its own backward needs. Those values are reference-counted handles to storage that already exists, so saving is cheap unless the tensor is later written in place. The graph is released as the backward pass consumes it.

What it costs. The design centre is large tensors, while pricing code is scalar-heavy, so per-operation dispatch overhead matters.

The abstraction layer - sketch

Template classes expose one syntax; specialisations dispatch to the backend.

```
template <class Backend> struct Vector { /* one syntax for the pricing code */ };
template <> struct Vector<EigenBackend> { Eigen::VectorXd data; /* ... */ };
template <> struct Vector<TorchBackend> { torch::Tensor data; /* ... */ };

template <class Backend> struct DiscountCurve { // all that pv() may know
    virtual Vector<Backend> df(const std::vector<Time>& t) const = 0;
};

// Written once, compiled for every backend. The schedule is not a risk factor.
template <class Backend> typename Backend::Scalar
pv(const std::vector<Time>& times, const Vector<Backend>& amounts,
   const DiscountCurve<Backend>& curve) {
    return sum(amounts * curve.df(times));
}
```

And then we want to calculte ρ. Curve and product code can be can be implemented independently. Adding new curve implementation doesn't require to maintain other code for AD to work. Switching a scenario to AD is a template parameter, not a rewrite.

How the project actually ran

- A series of prototypes, each removing one specific uncertainty, before any commitment to migrate
- Migration scenario by scenario, with reconciliation against the previous implementation at every step
- Finite differences kept as both the fallback and the independent check on AD numbers
- Unexpected technical problems appeared regularly and looked critical at the time. But all of them were eventually solved

Results

What we can report

Results

- **No full rewrite was needed.**
- **libtorch copes with genuinely complex code**, both in class structure and in the numerical algorithms.
- **Production runs stayed inside the same limits**, in time and in memory. That was the criterion that mattered
- **Effort: a few man-years**

A Note on benchmarking

the first production versions

A full like-for-like performance comparison is not an easy exercise. The two engines use different recording strategies, the code was restructured along the way, and the functional scope kept moving. What was critical for us is operational, not academic: the total time for **production** risk runs stayed within the same limits.

First version - the code is in dual-compatibility mode Many algorithms were written against the previous engine and ported to torch *to make them work*, not to make them fast. The abstraction layer that made the migration affordable also constrains each backend to a shared syntax. This is deliberate: it is what let us ship at all.

So the current numbers are a floor, not a ceiling. libtorch-specific optimisation is still in many cases an open opportunity.

References

What this talk leans on

- **Capriotti, Giles (2024).** *15 Years of Adjoint Algorithmic Differentiation in Finance.* Complexity model, the computational-graph figures, Monte Carlo, PDE and calibration applications.
- **Savine (2019).** *Modern Computational Finance: AAD and Parallel Simulations.* Wiley. Memory management, checkpointing, practical limits.
- **Andreasen, Savine (2018).** *Modern Computational Finance: Scripting for Derivatives and xVA.* Wiley.
- **Giles, Glasserman (2006).** *Smoking adjoints: fast evaluation of Greeks in Monte Carlo calculations.* Risk. Where this arrived in finance.
- **Beda, Korolev, Sukhikh, Frolova (1959).** *Programs for automatic differentiation for the machine BESM.* ITM & VT, Academy of Sciences, Moscow. The earliest known AD implementation.

Q&A

Thank you